Supplementary Proof Catalogue: Applying Normalized Systems Theory to Data Vault 2.0

Keywords: Normalized Systems Theory, Data Warehouses, Data Vault 2.0, Evolvability, Maintainability.

Abstract: Data warehouses must accommodate changing requirements without accumulating architectural complexity that impedes subsequent evolution. We propose a restricted Data Vault 2.0 architecture guided by Normalized Systems Theory, mapping warehouse structures and processes to normalized elements and defining rules that contain change propagation. Constructive proofs for a catalogue of anticipated warehouse changes identify affected implementation artifacts and establish bounds independent of accumulated warehouse size under explicit compatibility assumptions. The reference architecture prioritizes demonstrable evolvability and supports subsequent optimization and migration automation.

1 SCOPE AND STANDING PREMISES

This catalogue accompanies *Applying Normalized Systems Theory to Data Vault 2.0: A Constructive Approach to Evolvability*. It uses the final article's abstract, architectural restrictions R1–R7, and analytical model. P1–P9 reproduce its change-specific arguments; P3–P5 have separate subsections here. P10–P30 give further restricted constructions and conditional derivations. P18a additionally widens a canonical type through a new Satellite and a checked old interface; P19–P31 retain their identifiers. P31 addresses successive admitted changes, and Boundary Results 1–2 explain limits.

The main article is self-contained. This catalogue does not establish unrestricted coverage of every change in its catalogue: each result retains its own semantic, lifecycle, and implementation premises. The counts concern an analytical construction, not a deployed warehouse or experimental measurements. Table 1 locates the examples without replacing their assumptions.

1.1 Elements and Architectural Scope

Normalized Systems Theory (NST) distinguishes Data, Task, Connector, Flow, and Trigger elements. In this construction they encapsulate storage/access and configuration, individual actions, external interactions, state-driven sequencing, and activation, respectively. An element can contain several maintained artifacts. The associated concerns are Separation of Concerns (SoC), Data Version Transparency (DVT), Action Version Transparency (AVT), and Separation of States (SoS) (Mannaert et al., 2016, Chapters 12 and 15).

The boundary contains warehouse-owned acquisition, processing, configuration, and delivery interfaces, but not source applications or external report implementations. Published contracts remain obligations. A loading path is a target-specific projection through narrow Stage storage and a loader into one Vault target, with its interfaces, bindings, and registrations. Shared key-field bindings are not copied into every path.

The construction assumes identified batch observations. Link-on-Link dependencies and overloaded multi-source Satellites are excluded (Cuba, 2020, Chapter 4). Multi-active/tracking Satellites, computed-aggregate/exploration Links, and non-historized Links are outside the selected patterns, not classified as invalid (Linstedt and Olschimke, 2016, Chapter 5). Business Vault results are derived Satellites. Streaming, record-level deletion/erasure, general late arrivals, global rehashing, platform migration, and shared point-in-time/bridge maintenance have no general bound in this catalogue.

1.2 Architectural Restrictions

The following R1–R7 are the same restrictions as in the main article. They apply to transactional databases and ETL tools supporting the specified contracts and protocols and are sufficient construction choices rather than necessity claims.

R1: Add descriptive structures. Every added descriptive attribute receives a source-specific, one-attribute Satellite and narrow Stage path; parent keys

Table 1: Catalogue index and worked-example artifact bounds. Each value applies only to its specified construction and premises. P6 has general bound $4 + r$; its date example uses $r = 0$. P18a includes a new wide path, retirement of old acquisition, one checked old provider, and one new delivery interface. P21's example has no additional decoder or delivery artifacts. P23 assumes acquisition of the referenced Supplier identities already exists. P24 distinguishes retained versus withdrawn role fields.

Proof	Worked case	Bound	Proof	Worked case	Bound
P1	Source-table addition	32	P17	Recovery-response revision	3
P2	Logical feed retirement	2	P18	Compatible type widening	4
P3	Descriptive attribute addition	10	P18a	Canonical type widening	20
P4	Delivery-interface addition	4	P19	Exactly recoverable narrowing	4
P5	Leaf-interface retirement	2	P20	Three value-mapping rules	7
P6	Exact representation change	4	P21	New descriptive meaning	12
P7	Business-rule revision	12	P22	Hub-key version replacement	25
P8	New failure response	6	P23	New Link participant	10
P9	Lossless split, live contract	77	P24	Removed Link participant	$9/11$
P10	Source-table rename	2	P25	Split, retained history only	56
P11	Source-attribute rename	2	P26	Two-part horizontal split	46
P12	Descriptive field withdrawal	2	P27	Two-part horizontal merge	42
P13	Three-feed onboarding	99	P28	Lossless vertical split	46
P14	Three-feed withdrawal	7	P29	Lossless vertical join	42
P15	New business-rule calculation	12	P30	Compatible constraint change	≤ 1
P16	Compatible task activation	3	P31	Successive admitted changes	Eq. 49

and technical metadata are additional. Existing payloads/hashdiffs remain immutable. This is stricter than in-place extension (Cuba, 2020, Chapter 4) and trailing-NULL append-and-trim hashing (Linstedt and Olschimke, 2016, Section 11.2.5.1). Anchor Modeling provides the separate-attribute precedent (Rönnbäck et al., 2010, Section 5); NST does not require one-attribute Satellites.

R2: Preserve data and action contracts. Interfaces declare fields, types, identity, grain, time, and availability. Dependencies include nonreturned predicate, join, grouping, and expression inputs (Manousis et al., 2015, Section 2.1). No positional access, whole-schema inheritance, or private-layout access is permitted. Hub identity and Link grain are fixed per version; changed meaning requires a new structure and compatibility decision. Stable write ports coordinate concurrent writers without loader rewrites. Delivery bindings are independently retirable; shared providers remain.

R3: Separate tasks and orchestration. Source access, loading, rules, and control have separately replaceable implementations; shared functionality is accessed, not copied. The controller records state per task/version/batch and invokes tasks uniformly. Transitions are independent configuration records rather than a monolithic generated workflow (Mannaert et al., 2016, Section 15.2.4). Paths consume identified completed observations, preserving their identity on retry. Registration and publication follow dependencies and R7.

R4: Predefine source lifecycle. Each table feed has one control record with its identifier, lifecycle, and last completed batch. Every extraction path uses that record; the controller checks it before resolution/access. Inactive connectors are neither invoked nor revalidated against deleted relations. Consumers already distinguish current availability from retained history.

R5: Retain history without cascades. Source removal retains warehouse history, structures, loaders, and interfaces. Shared Hubs/Links are not retired with one contributing feed.

R6: Isolate source names and representations. Before Stage, an adaptation port resolves logical feed/field identifiers against an identified snapshot and decodes exactly, without business reconciliation. Paths and hashes use canonical identifiers/values. Snapshot acquisition/replay, registry, and resolver are existing parts of the initially conforming system.

Versioned relation bindings own physical table names and applicability. Independent field bindings own physical column names, canonical names/types, decoder formats, and R7 metadata. They reference logical feeds without copying table names. Resolution selects and pins applicable versions per batch. Changes append individual records, not complete schema copies or alias chains; replay uses retained bindings and captured data. Bindings are interpreted

at execution, not compiled into paths. New mechanism functionality requires separate task-change accounting. Labels, mapping history, and provenance trace source/warehouse naming drift. This specializes adapter-schema and source-entry repair ideas (Vassiliadis et al., 2023; Awiti et al., 2025).

R7: Predefine field lifecycle and availability. Every field binding has active/retired state, applicability, and a last-complete-observation reference from creation. The resolver checks lifecycle before physical access; retirement returns structured unavailability without reading the missing column. Supplied NULL, nonsupply, and retirement remain distinct; unexpected absence is not decommissioning. Feed retirement makes its fields unavailable without per-field edits.

Every public field-contract version admits `Unavailable(outOfRange)` from creation when a supplied non-null value lies outside its exactly representable domain. Checks are specific to the entity, field version, and observation: representable values remain exact; others are not capped or truncated. Read interfaces retain or derive each observation's status rather than return an earlier in-range value.

Tasks declare every required raw/derived input. Before publication, task/read wrappers apply one protocol to immediate providers for the requested observation: an unavailable required input yields unavailable output, never a stale result presented as current. The same protocol applies transitively through rules; independent paths continue. History retains observation times and cutoff metadata outside immutable payloads/hashdiffs. Retirement drains or cancels pending invocations, including retries, before its boundary. Replay uses captured data and retained mappings, not fresh publication. Bindings, interfaces, and tests instantiate this protocol; legacy retrofits are separately counted.

1.3 Proof Method and Running Example

Artifacts are independently maintained storage/interface definitions, operations, workflow registrations, test specifications, and configuration records. Each added, modified, or removed artifact is counted once. Runtime rows, invocations, and executions of unchanged tests are not artifacts. The counting follows NST's stability criterion (Mannaert et al., 2016, Section 12.1.1).

Let a warehouse be represented by $W = (V, E, \kappa, d, \gamma)$. Here V contains artifact identifiers; E declared dependencies; κ NST/verification roles; d maintained definitions, including configuration, dependency declarations, and interface specifications; and γ the resulting public contracts. Size is $N = |V|$. The reference class $\mathcal{A}$ satisfies R1–R7; immutability and retention also constrain transitions.

For example, let s be email-Satellite storage, i its read interface, and r an email-validation task. In this fragment,

$$V_{\text{ex}} = \{s, i, r\}, \qquad E_{\text{ex}} = \{(i, s), (r, i)\}. \tag{1}$$

An edge (a, b) means that a depends on b. Storage/interface artifacts have Data roles and r has a Task role; s, i belong to one Data-element realization. Definitions specify schema/read/validation logic, while $\gamma(i)$ promises a value or unavailable state for a customer and observation. Implementation may change while that promise remains satisfied. Loading/control artifacts are omitted from this three-artifact illustration.

A request Δ of kind τ has scope $s(\Delta)$, describing the requested change rather than warehouse size. For source-table addition, $s(\Delta) = (q, v, f)$ specifies loading paths, new Vault objects, and selected fields. The corresponding recipe defines a partial transformation T_Δ, applicable when its preconditions hold, producing $W' = T_\Delta(W)$. Its disjoint change sets are

$$\begin{aligned} V_+ &= V' \setminus V, \qquad V_- = V \setminus V', \\ V_\sim &= \{a \in V \cap V' : d'(a) \neq d(a)\}, \\ M(W, \Delta) &= |V_+| + |V_-| + |V_\sim|. \end{aligned} \tag{2}$$

Request-specific premises $\text{Pre}_\tau(W, \Delta)$ include faithful source mappings, authorized leaf retirement, or lossless decomposition. Their verification may require dependency or semantic analysis, whose cost is not bounded here.

Rename example. Suppose 100 paths use `customer_feed` and binding b_1 to `CRM.CUSTOMER`. A rename to `CRM.CLIENT` preserves identity, values, constraints, multiplicity, and observation semantics. Append binding b_2 and test specification t_2, retaining old definitions:

$$V_+ = \{b_2, t_2\}, \quad V_- = V_\sim = \varnothing, \quad M(W, \Delta_{\text{rename}}) = 2. \tag{3}$$

The existing resolver selects b_2 for later observations, while earlier observations retain b_1 and their captured data. No existing path changes. The two-artifact bound establishes structural locality, while correctness additionally requires that the renamed source preserve the existing data contract, including its business meaning.

Theorem 1 (Change-specific bounded impact)**.** *For each recipe P1–P9, let $K_\tau(s(\Delta))$ denote the upper*

bound obtained by counting its required architectural modifications. If $W \in \mathcal{A}$ and the recipe's premises hold, its transformation realizes the request, preserves existing contractual obligations and R1–R7, and satisfies $M(W,\Delta) \le K_\tau(s(\Delta))$. For fixed admissible scope and implementation patterns, the bound is independent of accumulated warehouse size.

Argument structure. The cases below identify their complete changed sets, justify preserved semantics, and check the restrictions. Additions use shared targets only when their contracts admit the new facts. Existing state-aware mechanisms interpret declared lifecycle/activation transitions without new caller branches. Replacement providers must preserve meaning and time, not only signatures. Each case states its additional premises. □

Scope is fixed before dependency analysis. Independent mappings, clauses, and generated edits count separately; a growing union, adapter, or test cannot be hidden as one artifact. Every live contract and reconstruction dependency must be identified. Growing adapter chains, parallel providers, or cloned paths are real costs (Manousis et al., 2015); forced repairs cannot be reclassified as new functionality. Initial generic mechanisms belong to the conforming architecture; retrofitting them is not included in a local recipe. Artifact counts concern definitions, not engineering effort.

1.4 Additional Premises for Supplementary Cases

The initial capture, resolver, registry, state controller, and availability mechanisms are part of the conforming warehouse; installing or extending them is not free work hidden in a local change. Every new field or rule must instantiate the same declared raw/derived-input availability protocol in its counted definitions. This allows a later withdrawal to affect runtime results without requiring a new branch in each dependent rule.

P26–P29 retain explicit source-correspondence conditions. Their stated bag equalities describe the required source-approved relationship between old and replacement observations; they are not conclusions about arbitrary source splits or joins. As in P9, these recipes also assume that existing Vault as-of interfaces reconstruct those valid snapshots at completed observation boundaries, including unchanged attributes with no new Satellite row. This is an explicit realization premise, not a storage-level theorem derived in this catalogue. Cutover retains old providers for earlier observations and introduces neither missing nor duplicate publication.

Source-system withdrawal in P14 remains a finite composition of table-feed retirements. A source-wide gate is not part of baseline R4. Likewise, P9 continues to require a non-null LegacyClientId for every Person admitted to its retained live contract and exactly one Account per Person. Multiple Accounts require a business-approved soft rule and separate accounting, as stated in P9; these are not additional cases silently included in its 77-artifact bound.

Table 2: P1 artifact counts; Branch has $q = v = f = 3$. Storage/access pairs belong to Data-element realizations.

Added definitions and responsibilities	Count
Feed lifecycle and relation binding (Data)	2
Initial field bindings (Data)	f
Logical-field projections (Connector)	q
Stage storage/read interfaces (Data)	$2q$
New Vault storage/read interfaces (Data)	$2v$
Loaders or write bindings (Task)	q
Extraction/loading registrations (Flow)	$2q$
Local contract-test specifications	q

2 CHANGE-SPECIFIC PROOFS

2.1 P1: Source-Table Addition

Request and premises. Integrate `CRM.BRANCH`, with `BranchId`, `Name`, and `Region`, using a new Hub and two one-attribute Satellites. Assume R1–R7, a configured source endpoint/resolver, supported decoders, and a supplied field/key/relationship mapping. Reused Hub/Link write ports keep their identities; readers either retain a declared source scope or admit the additional conforming facts. Reconciliation, new conversions, rules, and delivery adoption are separate requests.

Construction and bound. Let q count target-specific loading paths, $v \le q$ new Vault objects, and f distinct selected fields, including keys. Create a feed lifecycle record, relation binding, and f initially active field bindings. Each path selects logical identifiers into its narrow Stage output. Shared key bindings are created once. Table 2 lists the complete recipe; existing snapshot/resolver/state mechanisms and decoders are reused. Consequently,

$$\begin{aligned} M(W,\Delta_{\text{table}+}) &\le 2+f+q+2q+2v+q+2q+q \\ &= K_{\text{table}+} = 2+f+7q+2v. \end{aligned} \tag{4}$$

Branch gives $2+3+21+6=32$, including its relation and three field bindings. Any extra independently maintained mapping/check adds to the count.

Locality and preservation. New source bindings change no existing path's physical references. New descriptive payloads/hashes do not alter old ones (R1); reused write ports coordinate additional writers and preserve the admitted identity/population contract (R2). The controller interprets registrations rather than rewriting a source union (R3), and each path uses the existing lifecycle protocol (R4). History remains (R5). Binding versions are pinned (R6); field declarations, interfaces, and tests instantiate R7 from creation. Thus later renaming or withdrawal needs no new per-path behavior. For fixed q, v, f, existing warehouse size and unrelated paths contribute zero edits. This applies NST's separation and interface reasoning (Mannaert et al., 2016, Section 15.3.3).

2.2 P2: Logical Source-Table Retirement

Request and premises. Stop acquiring data from a retired customer source table while retaining its warehouse history and shared Hub. Assume R1–R7 and explicit decommissioning. Before retirement takes effect, the controller ensures that pending extractions and retries either complete or are cancelled without publishing incomplete batches as successful.

Before each extraction or retry, the controller checks the feed's shared lifecycle record. Only active feeds are resolved to physical source tables and accessed. Thus, retiring a feed stops all dependent extraction paths without modifying their definitions. Historical queries use retained warehouse data, while downstream contracts already support unavailable current inputs.

Construction and bound. Retire the feed-control record and add one local test specification:

$$M(W, \Delta_{\text{delete}}) \leq 1_{\text{control}} + 1_{\text{test}} = K_{\text{delete}} = 2. \quad (5)$$

The update is Data configuration; the unchanged controller stops dispatching the Connector. Stage/Vault structures, loaders, interfaces, Flow definitions, and Trigger remain. Tasks requiring fresh input follow the already specified unavailable-input behavior.

Locality and preservation. All paths share the feed identifier, so one lifecycle change replaces per-job disabling. Neither retained history nor the old published availability behavior requires new caller code. Even 100 Satellite paths and 20 delivery interfaces need no definition edits, although their runtime outputs can be affected. R1–R2 remain unchanged, R3–R4 implement an admitted transition, and R5–R6 retain history and mappings without live-source access. R7 derives field availability from the feed; no field records are rewritten and the last batch is not republished as fresh. This specializes state separation (Mannaert et al., 2016, Section 12.2.4), not physical erasure, source replacement, or continued production of missing facts.

2.3 P3: Descriptive Attribute Addition

An active customer feed supplies `EmailAddress`; its parent key, decoder, source access, and unused-field tolerance already exist. The request captures future email history without requiring old delivery contracts to adopt it. Create a one-attribute Satellite and its Stage/loading path, reusing the feed gate, relation binding, and key mapping. The definitions are one initially active field binding, one logical-field projection, two Stage storage/access artifacts, two Satellite storage/access artifacts, one loader, two Flow registrations, and one test:

$$\begin{aligned} M(W, \Delta_{\text{attribute}+}) &\leq 1+1+2+2+1+2+1 \\ &= K_{\text{attribute}+} = 10. \end{aligned} \quad (6)$$

Old projections ignore the new field. No shared payload, hashdiff, or all-attribute delivery definition changes. The new immutable target and contracts preserve R1–R2; registered tasks and the existing gate preserve R3–R4; history remains under R5. R6–R7 are initialized in the counted binding, interfaces, and test. The Trigger is unchanged. A requested delivery adoption is counted separately.

2.4 P4: Delivery-Interface Addition

Publish a report-facing customer-name interface over fields already provided by one public contract. Assume a bounded projection, no new business meaning or missing inputs, and existing request/state and publication mechanisms. Add its read contract (Data), projection (Task), publication binding (Connector configuration), and test:

$$M(W, \Delta_{\text{delivery}+}) \leq 1+1+1+1 = K_{\text{delivery}+} = 4. \quad (7)$$

The new reader alters neither the provider nor existing readers. It preserves input identity, time, and availability; its binding is independently retirable. Thus R1–R7 remain satisfied without changing Flow or Trigger. This is on-demand delivery; materialized output, additional joins, or scheduling adds separately maintained artifacts.

2.5 P5: Delivery-Interface Retirement

Withdraw one interface whose lifecycle permits retirement, after all previously accepted requests have completed. It is a *leaf*: no warehouse task uses its output. Retire its publication binding and add a local test:

$$M(W, \Delta_{\text{delivery}-}) \leq 1 + 1 = K_{\text{delivery}-} = 2. \quad (8)$$

The existing Connector declines new requests through its predeclared retired state. Shared providers and history remain; no downstream warehouse definition requires repair. Consequently R1–R7 are unchanged. External report migration is not included. If another warehouse task consumes the interface, the leaf premise fails and its contractual obligation must instead be preserved or migrated.

2.6 P6: Exact Source-Representation Change

Request and premises. A source changes birth dates from `DD/MM/YYYY` to `YYYY-MM-DD`; the canonical date domain, meaning, null policy, parent key, grain, and observation time are unchanged. Under R1–R7, one owned decoding task supplies the field to dependent paths. The source identifies the format explicitly, not by guessing ambiguous dates, and faithfully encodes every required canonical value. Invocations pin decoder versions; audit-required original representations remain in the existing capture.

Construction, semantics, and bound. Append a field/format mapping, decoder task, format-to-decoder binding, and local test specification; retain earlier versions. Let D be the admitted canonical domain, $e_1 : D \to S_1$ the new encoding, and $g_1 : S_1 \to D$ the decoder on valid inputs. Require

$$g_1(e_1(x)) = x \quad (x \in D). \quad (9)$$

Each downstream computation F therefore receives the same value: $F(g_1(e_1(x))) = F(x)$. Canonical hash inputs/order remain unchanged, so formatting alone creates no descriptive change. For r independently specified value-mapping rules,

$$\begin{aligned} M(W, \Delta_{\text{repr}}) &\leq 1_{\text{mapping}} + 1_{\text{decoder}} + 1_{\text{binding}} + 1_{\text{test}} + r \\ &= K_{\text{repr}} = 4 + r. \end{aligned} \quad (10)$$

The date example has $r = 0$: four artifacts for five or 500 readers.

Locality and preservation. Decoders map directly to the canonical contract, not through a growing chain. Storage, loaders, and hashes remain (R1–R2); pinned invocation and gate protocols remain (R3–R4); previous data/decoders remain (R5). The mapping stays source-owned (R6), and decoding applies only to available input under unchanged R7. Flow/Trigger definitions do not change. Exact decoding and connector SoC preserve the data contract; an unchanged field name alone would not (Mannaert et al., 2016, Sections 12.2.1–12.2.2).

Rename specialization and limits. A source-confirmed physical table/field rename preserving values, identity, constraints, multiplicity, and time needs one new binding version and one test, hence $K_{\text{rename}} = 2$. Relation bindings alone own table names; field bindings refer to logical feeds. A field rename leaves its canonical name, decoder, and lifecycle unchanged. Old batches retain old mappings; paths are not regenerated. Simultaneous table and field renaming uses two such pairs, bounded by four.

Widening is covered only if the public domain admits all arriving values. A lossy encoding mapping both 1.2 and 1.8 to 1 has no exact inverse. Lost information or changed category meaning requires additional data or a different versioned contract, not this bound.

2.7 P7: Business-Rule Revision

Request and premises. Replace "purchase within 90 days" by "contact within 60 days and consent" as the active-customer policy. Required raw attributes and R1–R7 already exist. The new rule has two predicates and one window parameter. Public inputs are stable entity/observation references; each policy owns its required-field projection. Callers either request a fixed policy or intentionally follow the policy applicable at a specified evaluation time, rather than copy formulas or thresholds.

Effective-dated policy records select and pin the implementation, parameter, and input batch. Result storage is policy-specific and publication follows completion. New applicability begins after already published evaluation times; retroactive reinterpretation is separate. External inputs, if any, are captured before evaluation.

Construction and count. Add the projection, rule task, parameter, a narrow result buffer with read access, a one-attribute Business Vault result Satellite with read access, loader, two Flow registrations, effective policy binding, and local test. Earlier tasks, parameters, bindings, and results remain. The bound

is

$$\begin{aligned} M(W,\Delta_{\text{policy}}) \le\ & 1_{\text{input}} + 1_{\text{rule}} + 1_{\text{parameter}} \\ & + 2_{\text{buffer}} + 2_{\text{result}} + 1_{\text{loader}} \\ & + 2_{\text{flow}} + 1_{\text{binding}} + 1_{\text{test}} \\ =\ & K_{\text{policy}} = 12. \end{aligned} \tag{11}$$

Evaluation and persistence are separate Tasks; pairs count Data storage/access. No existing delivery or schedule changes.

Semantics and preservation. Let x identify a captured entity observation, t the evaluation time, v an explicit policy, and $f_v(x,t)$ its result. For the applicable version selector $\rho(t)$, the existing interfaces are

$$\begin{aligned} \text{evaluate}(v,x,t) &= f_v(x,t), \\ \text{evaluateCurrent}(x,t) &= f_{\rho(t)}(x,t). \end{aligned} \tag{12}$$

A recent purchaser without consent may change from true to false for policy-following callers; fixed-version callers retain their meaning. Only the selected computation changes, not the invocation mechanism.

The immutable result satisfies R1; explicit policy promises satisfy R2. Pinned, stateful tasks and input availability preserve R3–R4; earlier results remain (R5); inputs are canonical (R6). Unavailable consent is not false: R7's existing wrappers propagate unavailability without rule or downstream-caller edits. This applies AVT and separated rule concerns (Mannaert et al., 2016, Section 15.3.3) (Cuba, 2020, Chapter 1). Extra clauses, inputs, parameters, or outputs add counted artifacts. P7 preserves results produced under earlier policy versions; retroactively replacing their meaning is outside its scope.

2.8 P8: Extending Task Failure Handling Without Changing Callers

Request and premises. Activate a compatible `LookupSourceRecord` implementation for future invocations. It can report temporary service unavailability, to be handled by the existing retry mechanism. Under R1–R7, the result envelope already contains invocation/input/version identity, completion, an extensible diagnostic reason, and retry information. Callers interpret completion/non-completion, not an exhaustive reason list. A separate controller selects responses. Attempt limits, backoff, idempotent writes, and retained batch identity already exist.

Construction and count. Add the replacement Task and update its single activation record. Add a recovery Task, a reason-to-handler registration, and two tests for the lookup contract and response. For example, `batch_42` records "not complete: temporary service unavailable"; the controller selects the handler, which requests a permitted retry. Dependent output is not published complete before success.

$$\begin{aligned} M(W,\Delta_{\text{recovery}}) \le\ & 1_{\text{implementation}} + 1_{\text{activation}} + 1_{\text{handler}} \\ & + 1_{\text{response}} + 2_{\text{tests}} = K_{\text{recovery}} = 6. \end{aligned} \tag{13}$$

Activation is Data configuration and response selection a Flow responsibility; Trigger and payload definitions remain.

Locality and preservation. For n callers let c_i be edits forced by the diagnostic reason. Each receives the existing non-complete state and delegates recovery; therefore $c_i = 0$ and $M \le 6 + \sum_{i=1}^{n} c_i = 6$. This remains six for 80 callers. Caller-specific branches would violate the premise and NST's state-separation mechanism (Mannaert et al., 2016, Section 12.2.4).

R1–R2 remain unchanged; action/result contracts and separate recovery preserve R3. Retries respect source/field gates (R4, R7), while retirement drains/cancels pending invocations before its boundary. A retired column is not treated as a temporary outage. R5–R6 retain old implementations, captured inputs, and pinned mappings. Extra diagnostic structures or responses count separately. Structural locality neither guarantees successful recovery from every outage nor proves exactly-once execution on an arbitrary platform.

2.9 P9: Lossless Split with Live Compatibility

Request and premises. Split `CLIENT` (ClientId, Name, Balance) into `PERSON` (PersonId, LegacyClientId, Name) and `ACCOUNT` (AccountId, PersonId, Balance). Acquire the replacement model while preserving one live `client_v1` read contract with the original identity and one-row-per-client grain.

Besides R1–R7, assume a source-approved lossless mapping covering exactly the required client population. At each supported boundary, PersonId, LegacyClientId, and AccountId are keys; LegacyClientId is non-null and preserves the old identity. Every Account references an admitted Person, and each Person has exactly one Account. Identity correspondences are fixed, although new identities may be appended. Reassignment and row withdrawal require separate temporal handling; field meanings and null/time rules are unchanged.

Inputs are coherent completed observations, not arbitrary latest batches. Person ingestion supplies

its Hub instance before the Account Link is published. Cutover has neither gaps nor duplicate publication. Other retained contracts permit historical-only service; additional live obligations are counted. Provider records select by observation boundary without nested compatibility chains.

Construction. Retain the existing Client Hub and historical Satellites. Person ingestion loads LegacyClientId into the existing Client Hub, inserting only previously unseen keys, and introduces a Person Hub, Name Satellite, and Client–Person Link: four paths, three new targets. Account ingestion introduces an Account Hub, Person–Account Link, and Balance Satellite: three paths and three new targets. The replacement feeds do not populate the old Client Satellites. Links record approved correspondences without Link-on-Link references (Cuba, 2020, Chapter 4).

A Business Vault compatibility Task follows these relationships, combines Name and Balance observations, and projects the old fields. Its read interface and effective provider record serve `client_v1` after cutover; earlier observations retain their old provider. Retire the old feed when its final batch and the first coherent replacement inputs are ready.

Read-contract argument. At one boundary, let P,A be Person/Account snapshot bags. Write p.id, p.legacy, p.name, a.person, and a.balance for their values. The reconstruction G is

$$G(P,A) = \big[\,(p.\text{legacy}, p.\text{name}, a.\text{balance}) : p \in P,\ a \in A,\ a.\text{person} = p.\text{id}\,\big]. \tag{14}$$

Bag semantics retains multiplicity. For every admitted Person p,

$$\big|\{a \in A : a.\text{person} = p.\text{id}\}\big| = 1. \tag{15}$$

Each Person contributes one row; unique LegacyClientId supplies the old key, and the approved mapping preserves values and population. Thus $G(P_b,A_b) = \mathit{CLIENT}_b$ as bags at each supported boundary b.

This assumes the existing Vault as-of interface returns valid values at completed boundaries, including unchanged attributes with no new Satellite row; incomplete loads are not exposed as complete. It is a realization premise, not a derived storage-level theorem. The result concerns read compatibility rather than bidirectional data independence (Hermann et al., 2018, Section 5).

If the refactoring also permits multiple Accounts per Person, the business must define a soft rule for the old interface: for example, select the oldest/newest account or sum all balances. This extends the one-account case and requires separate semantic justification and artifact accounting.

Footprint and preservation. Each replacement feed has three selected fields and requires lifecycle and relation bindings. Seven paths and six new targets give

$$K_{\text{raw}} = 2\cdot 2 + (3+3) + 7\cdot 7 + 2\cdot 6 = 71. \tag{16}$$

Add four compatibility artifacts: Task, read contract, effective provider record, and test. On-demand invocation requires no new result store or schedule. Old-feed retirement adds two:

$$M(W,\Delta_{\text{split}}) \le 71 + 4 + 2 = K_{\text{split}} = 77. \tag{17}$$

For L distinct live providers using this four-artifact pattern, the bound is $K_{\text{raw}} + 4L + 2$. More complex providers require their own artifact-count breakdowns. Since L is an existing obligation, uniformity requires bounded provider count and reconstruction size; neither follows automatically from request scope.

New payloads satisfy R1; identity and reconstruction preserve R2. Registered tasks/gates preserve R3–R4, history remains (R5), source mappings are owned (R6), and R7 prevents unavailable inputs from yielding fresh Client results. Later refactorings must satisfy the same semantic and reconstruction premises; additional joins or providers forced by accumulated contracts remain counted costs.

2.10 P10: Rename a Source Table

Request and premises. The physical customer relation is renamed. Its former name is `CRM.CUSTOMER`; its replacement is `CRM.CUSTOMER_DETAILS`. The logical identifier `customer_feed` stays the same. The source confirms that identity, values, constraints, multiplicity, and observation boundaries are unchanged. R1–R7 already hold, including runtime resolution and retained mapping selections. This is a supplied correspondence, not automatic discovery of semantic equivalence.

Construction and count. Append one relation-binding version containing the new physical name and its applicability boundary. Add one local contract-test specification comparing the admitted old and new projections. Do not modify field bindings: they reference the logical feed. Let N count existing artifacts, including mapping versions; $\Delta_{\text{tableName}}$ be this rename; $M(W,\Delta_{\text{tableName}})$ count distinct additions, modifications, and retirements; and $K_{\text{tableName}}$ be the recipe bound.

$$M(W,\Delta_{\text{tableName}}) \le 1_{\text{relation binding}} + 1_{\text{test}} = K_{\text{tableName}} = 2. \tag{18}$$

The binding is Data configuration supporting the source Connector. Existing Data payloads, loading Tasks, Flow registrations, and Triggers require zero edits.

Why it remains local. Every extraction selects the unchanged logical feed. The resolver looks up its one applicable physical binding, so ten or a thousand paths use the same new name without rewriting those paths. The field-level mappings are still valid because no field representation changed. Recompiling all paths into new persistent definitions would violate the runtime-resolution premise and must be counted, not hidden as one metadata change.

Preservation and example. For a feed used by 100 Satellite paths, the recipe still changes two artifacts. Already accepted batches retain the original binding; new batches pin the admitted replacement; replay uses captured data, not the obsolete relation. Payloads and key/hash inputs stay fixed (R1–R2), stateful activation and source gates remain effective (R3–R4), and old history remains (R5). Direct, independently versioned name resolution preserves R6. Field lifecycle metadata and required-input handling remain unchanged (R7). The result covers physical renaming, not a different table reusing the former name.

2.11 P11: Rename a Source Attribute

Request and premises. The source changes one field label. It was `FullName`; it is now `DisplayName`. The warehouse still uses logical field `customer_name`. The admitted values and their business meaning, type, null policy, identity, and time interpretation are unchanged. R1–R7 already provide an independently versioned binding for this field; no consumer contains its physical spelling.

Construction and count. Append one field-binding version using the new physical name, the existing canonical name and type and decoder, and its applicability boundary. Add one mapping-contract test specification. Let N include all installed and retained artifacts; $\Delta_{\text{fieldName}}$ denote the rename; $M(W, \Delta_{\text{fieldName}})$ count distinct artifact changes; and $K_{\text{fieldName}}$ be the bound.

$$M(W, \Delta_{\text{fieldName}}) \leq 1_{\text{field binding}} + 1_{\text{test}} = K_{\text{fieldName}} = 2. \tag{19}$$

Data configuration in the Connector changes. Stage/Satellite storage, hashdiffs, Tasks, Flows, and Triggers do not. A simultaneous table and field rename composes two independent recipes, with conservative bound $2+2=4$.

Why it remains local. Paths request the logical field, not the source label. The unchanged decoder supplies the same canonical values. Existing filters, joins, hashes, and delivery computations therefore retain both their definitions and their required input meaning. The argument includes users of the field in predicates, not only users returning it in their output.

Preservation and example. For 250 existing users, appending one mapping and one test still costs two architectural changes. Old snapshots retain the old mapping selection; future invocations use the new record. Immutable payloads and contracts, state/gate handling, historical access, and direct mapping resolution preserve R1–R7. The new mapping retains R7's lifecycle and availability protocol. A name now attached to a different business meaning does not satisfy the premise; a maintained label alone is not DVT.

2.12 P12: Withdraw a Descriptive Attribute

Request and baseline premises. The source stops supplying `EmailAddress`, while the customer feed and other selected attributes remain active. Preserve email history and report its withdrawal; no current replacement value is required. The field is descriptive, not part of a Hub identity or a Link's participant key.

Assume R1–R7. In particular, each selected field already has a versioned lifecycle binding. The resolver checks that binding before physical access, and task/read-interface wrappers already interpret unavailable required inputs. This includes fields used only in filters, joins, or grouping. Their contracts distinguish retirement from a supplied NULL and from missing data without a decommissioning decision.

Retirement occurs at an approved boundary. The final accepted field observation is complete; any remaining invocations, including retries, finish or are cancelled through R7 before that boundary. Cancelled or incomplete work is not published as successful. The prior last-complete observation is known. These premises belong to the baseline; this operation does not install field-lifecycle support in legacy consumers or invent a value for an unconditional current-data promise.

Construction and count. Append one retired version of the email binding, recording its applicability boundary and the prior completed-observation reference, and add one parameterized retirement-test specification. Retain the Satellite payload, hashdiff, loader, historical rows, and old binding versions. No

separate per-consumer status definition or data backfill is introduced.

Let N count all existing warehouse artifacts including retained versions; $\Delta_{\text{attr}-}$ denote this descriptive withdrawal; $M(W, \Delta_{\text{attr}-})$ count distinct artifacts added, modified, or logically retired; and $K_{\text{attr}-}$ be the construction bound.

$$M(W, \Delta_{\text{attr}-}) \leq 1_{\text{field binding}} + 1_{\text{test}} = K_{\text{attr}-} = 2. \quad (20)$$

The field record is Data configuration interpreted by the Connector and Flow. Existing Data storage, Task definitions, and the Trigger do not change. Further independent platform definitions would increase the artifact-count breakdown; the stated recipe uses the existing R7 protocol.

Why it remains local. For subsequent input batches, the resolver returns the existing retired/unavailable state without resolving the absent column. Its narrow Satellite path therefore acquires no fictitious new email observation. Other fields use their own active mappings. Dependent computations follow the already implemented required-input checks and return unavailable rather than treating email as zero, false, or a confirmed business deletion. Historical access retains the last accepted observation with its time and cutoff.

These responses are consequences of interpreted field bindings and predeclared input dependencies. No loader, predicate, or delivery definition receives a new email-specific branch. The same protocol applies transitively when a rule consumes another rule's output: each checks its immediate required providers for the requested observation. Runtime checks may visit many elements, but their definitions are unchanged. Hence their number does not enter $K_{\text{attr}-}$.

Preservation and example. With 100 other selected fields and 20 dependent delivery interfaces, the recipe still changes only the binding and test. A phone path continues because its own required bindings remain active. Retained data and fixed payloads preserve R1–R2; stateful readiness and feed checks preserve R3–R4; history and direct mapping resolution preserve R5–R6. The declared field-lifecycle transition and unchanged availability wrappers preserve R7. Historical replay selects retained versions and captured input, without publishing it as fresh.

This proof establishes local withdrawal under the baseline contract, not recovery of a future email that the source no longer supplies. It is distinct from retiring the entire source table.

2.13 P13: Onboard a Source System

Request and premises. Integrate a supplied list of m source-table feeds from a new system. The supported connector technology and generic source resolver already exist. The approved integration specification states every selected field, key, relationship, and newly required target. R1–R7 hold, including the source-coverage promises of reused Hubs and Links. Cross-source reconciliation is not implicit.

For table i, let q_i count target-specific paths, $v_i \leq q_i$ count new Vault objects, and f_i count selected source fields. Let c_+ count all additional source-level connection, configuration, and test definitions.

Construction and proof. Install the c_+ source-level definitions. For each table, apply the table-addition construction: lifecycle and relation bindings, f_i field bindings, q_i narrow paths, and v_i target structures. Complete prerequisites before publishing a path; shared objects are created once. New records do not rewrite any old source projection or a universal source-union loader.

Let N count existing artifacts including versions; $\Delta_{\text{system}+}$ be the specified onboarding; $M(W, \Delta_{\text{system}+})$ count distinct artifact changes; and $K_{\text{system}+}$ denote the summed bound. A union of changed sets cannot exceed the sum of their sizes:

$$\begin{aligned} M(W, \Delta_{\text{system}+}) &\leq c_+ + \sum_{i=1}^{m} (2 + f_i + 7q_i + 2v_i) \\ &= K_{\text{system}+}. \end{aligned} \quad (21)$$

Here the Connector gains source access and logical projections; Data gains bindings, Stage structures, and required Vault objects; Tasks and Flows gain their local load definitions and registrations. Existing Triggers and consumers retain their definitions.

Numerical example and preservation. Take three independent Branch-like tables with $q_i = v_i = f_i = 3$. Let $c_+ = 3$: a connection descriptor, a credential-reference configuration, and its local test specification. Each table costs at most 32, so $K_{\text{system}+} = 3 + 3(32) = 99$. Credential values are not new copied loader definitions.

Each component construction preserves R1–R7; a dependency-aware order therefore preserves them for the onboarding as a whole. For fixed m, c_+, q_i, v_i, f_i, the bound is independent of N. It is not constant in the number of newly requested tables. A previously unsupported connector driver adds its explicitly specified implementation footprint rather than being free.

2.14 P14: Withdraw a Source System

Request and premises. Retire an explicitly supplied list of m table feeds of a source system, with historical access retained. R1–R7 hold; each listed feed has completed its final accepted batch, pending extractions and retries have completed or been cancelled, and all remaining consumers already handle its cutoff and unavailability. Let c_- count additional source-level changes, such as disabling an existing connection after its feeds have been retired.

Construction and proof. Apply the source-table retirement recipe to each listed feed: update its lifecycle record and add its local test specification. Then apply the c_- source-level changes. Do not delete shared Vault objects or rewrite every pipeline reading their history. The stateful controller interprets the existing per-feed gates.

Let N count installed and retained artifacts; $\Delta_{\text{system}-}$ be withdrawal of the specified list; $M(W,\Delta_{\text{system}-})$ count distinct artifact changes; and $K_{\text{system}-}$ be the bound. Then

$$M(W,\Delta_{\text{system}-}) \le c_- + \sum_{i=1}^{m} 2 = c_- + 2m = K_{\text{system}-}. \tag{22}$$

Lifecycle Data configurations change. Existing Connectors become inactive through those records. Data payloads, Tasks, Flows, and the common Trigger require no definition edits. The artifact-count breakdown counts each table's local verification specification, not one unbounded test program pretending to verify all feeds for free.

Numerical example and scope. For three designated feeds and one connection-state update, $K_{\text{system}-} = 1 + 2(3) = 7$. This bound is independent of unrelated warehouse artifacts, but depends on the supplied retirement list.

If "retire this source system" means all of its feeds, and that source accumulates arbitrarily many feeds over time, this recipe has impact growing with that population. It is not a uniform bound for that alternative elementary requirement. A preinstalled source-wide gate could support a different constant-size recipe, but is an additional architecture and proof, not assumed here.

Preservation. Every listed table reaches the already supported retired state without changing payloads or identities. Historical data and binding versions remain, inactive feeds are not resolved, and unlisted sources continue. Thus the finite composition preserves R1–R7, subject to the same availability premises as each table retirement.

2.15 P15: Introduce a Business-Rule Calculation

Request and premises. Introduce a `LargeBalance` calculation defined by comparing a customer's available balance with one threshold parameter. The parent identity and required canonical inputs already exist. This is a new policy/output, not a replacement of an old fixed business meaning. R1–R7 and the generic policy binding, result-state, and loading mechanisms already hold.

The rule declares every required input, whether a source-derived field or another business rule's output. Its input and result interfaces use R7's availability protocol for the requested observation: a missing required input yields an unavailable result, not false or a stale value presented as current. These declarations and checks belong to the counted projection, interfaces, and test.

The selected rule is one bounded comparison over specified inputs. A larger formula, additional parameters, extra raw inputs, or additional output attributes must be counted separately. No old report is implicitly required to adopt it.

Construction and count. Create the rule's input projection, evaluation task, threshold record, a narrow output buffer and its read interface, and a new one-attribute Business Vault result Satellite and its interface. Add its loader, two flow registrations, a logical policy binding, and one local contract-test specification.

Let N include existing and retained artifacts; $\Delta_{\text{rule}+}$ be this introduction; $M(W,\Delta_{\text{rule}+})$ count distinct artifact changes; and $K_{\text{rule}+}$ be its bound.

$$\begin{aligned} M(W,\Delta_{\text{rule}+}) &\le 1_{\text{input}} + 1_{\text{rule}} + 1_{\text{parameter}} \\ &\quad + 2_{\text{buffer}} + 2_{\text{result}} + 1_{\text{loader}} \\ &\quad + 2_{\text{flow}} + 1_{\text{binding}} + 1_{\text{test}} \\ &= K_{\text{rule}+} = 12. \end{aligned} \tag{23}$$

The buffer, result, parameter and binding are Data responsibilities; calculation and persistence are separate Tasks; the two Flow records sequence them. Source Connectors and existing Triggers are unchanged. A new external delivery product is additional.

Why it remains local. The rule reads public inputs without revising their semantics. Its output and binding are new, so no earlier policy must be edited and

no consumer's copied formula must be replaced. The controller already supports evaluation and loading as version-pinned tasks. A threshold of 10,000 is one parameter record, not a warehouse-wide configuration replacement.

Preservation and example. The numerical example requires 12 artifacts for 10 or 1,000 unrelated policies. New output history has a fixed descriptive payload; inputs, identity, task ports, and availability contracts are preserved. Stateful evaluation, source gates, and source-name indirection remain unchanged. Old rule code/results remain reachable. If a required input is later withdrawn under R7, the rule and its downstream consumers already propagate unavailability without new definition changes. Hence R1–R7 continue to hold. The proof establishes a bounded implementation construction for the supplied rule; it does not prove the business appropriateness of the threshold.

2.16 P16: Activate a Compatible Task Implementation

Request and premises. Replace `LookupSourceRecord_v1` by a bounded `LookupSourceRecord_v2` for future calls through the existing logical task name. The new implementation satisfies the same promised input/output behavior and completion/failure contract; it does not add a new recovery protocol or business meaning. R1–R7 and the effective provider registry already exist. Current invocations retain their pinned implementation.

Construction and count. Add the replacement Task implementation, append or change one activation record with its applicability boundary, and add one local compatibility-test specification. Retain the old version for already accepted invocations and required replay.

Let N count existing artifacts and versions, Δ_{activate} be this replacement, $M(W,\Delta_{\text{activate}})$ count distinct artifact changes, and K_{activate} be the recipe bound.

$$\begin{aligned} M(W,\Delta_{\text{activate}}) &\le 1_{\text{implementation}} \\ &\quad + 1_{\text{activation}} + 1_{\text{test}} \\ &= K_{\text{activate}} = 3. \end{aligned} \tag{24}$$

Activation is Data configuration governing the Task. Existing data storage, source/delivery Connectors, caller Flow definitions, and Triggers do not change.

Why it remains local. Suppose n workflows invoke the logical task. Each already resolves the effective provider through the shared calling mechanism. If c_j is the number of edits forced on caller j solely by this replacement, the unchanged contract and runtime selection give $c_j = 0$. Thus

$$M(W,\Delta_{\text{activate}}) \le 3 + \sum_{j=1}^{n} c_j = 3. \tag{25}$$

For 80 callers the bound is still three. Explicitly pinned old versions are not silently repointed; a requirement to alter every historical invocation is different.

Preservation and limit. The implementation keeps the canonical contract and all required states, uses the same source gate and pinned mappings, and preserves previous code and observations. R1–R7 therefore remain valid. A task name cannot hide arbitrary implementation growth: additional independently maintained helpers or clauses extend the supplied change scope and the artifact-count breakdown. New error responses are handled by the separate recovery construction, rather than assumed compatible from an unchanged signature.

2.17 P17: Revise an Existing Recovery Response

Request and premises. An existing response to temporary unavailability is replaced by a bounded implementation that selects the controller's already supported short-backoff option rather than its long-backoff option. Both options are permitted by the existing response contract. Diagnostic reasons, completion states, attempt limits, input identity, and retry/write semantics remain unchanged.

Assume R1–R7, the extensible result envelope, separately versioned response records, and the existing backoff options. This change does not install a new controller capability or change every caller's contract.

Construction and count. Add the replacement recovery Task, add the effective reason-to-handler binding, and add one local response-test specification. Retain previous bindings and pin a handler selection for each affected invocation as specified by the existing controller protocol. Let N count existing artifacts and versions, Δ_{response} denote this revision, $M(W,\Delta_{\text{response}})$ count distinct artifact changes, and

K_{response} be the bound:

$$M(W, \Delta_{\text{response}}) \leq 1_{\text{handler}} + 1_{\text{binding}} + 1_{\text{test}} = K_{\text{response}} = 3. \quad (26)$$

Why it remains local. Callers report non-completion through the same envelope and do not select recovery implementations themselves. The controller interprets the one new response record, while the handler uses an existing retry action. Payload Data and Connector contracts do not change; no caller Flow or Trigger gains a new branch. The selected backoff is part of the local handler's bounded implementation. A new independently maintained parameter record, if required by another implementation, adds to the count.

Preservation and example. For 80 invokers, changing three artifacts suffices, just as for one. Retry still uses the same logical input batch, pinned source mapping, and idempotent write protocol; it rechecks source and field lifecycle before source access. Pending invocations are completed or cancelled before a coordinated retirement boundary; recovery cannot reopen a retired source or column. Existing storage, interfaces, history, and R1–R7 remain valid.

Changing the result-state schema, inventing a new final failure meaning, or promising a different externally observable retry guarantee is outside this recipe. Such a request cannot be made local merely by labelling it a "recovery revision".

2.18 P18: Widen a Source Type without Changing the Canonical Contract

Request and premises. A source changes an integer representation from a narrower to a wider type, or increases a string length. The warehouse field's existing canonical domain already represents every arriving value. Its identity, units, null behavior, ordering where relevant, and temporal interpretation are unchanged. R1–R7 hold, and the source declares the new format rather than requiring callers to guess it.

This is representation widening, not automatic widening of every published interface. If new values exceed an existing consumer's promised domain, this proof does not apply to that consumer.

Semantic argument and construction. Let D be the existing canonical domain, $e_w : D \to S_w$ the new encoding, and $g_w : S_w \to D$ its decoder on valid inputs. Require $g_w(e_w(x)) = x$ for every admitted $x \in D$. Consequently any unchanged computation F still receives $F(g_w(e_w(x))) = F(x)$. Add the new decoder, a field/format-binding version, a format-to-decoder registration, and its local test specification. The same canonical serialization reaches existing hashes.

Let N count all existing and retained artifacts; Δ_{widen} denote this widening; $M(W, \Delta_{\text{widen}})$ count distinct artifact changes; and K_{widen} be the bound. This recipe has no additional value-mapping rules:

$$M(W, \Delta_{\text{widen}}) \leq 1_{\text{mapping}} + 1_{\text{decoder}} + 1_{\text{registration}} + 1_{\text{test}} = K_{\text{widen}} = 4. \quad (27)$$

Mapping and activation records are Data in the source Connector; the decoder is a Task. Existing storage, Flows, and Triggers are unchanged.

Example, locality, and preservation. Suppose the warehouse contract is already a sufficiently wide integer and the source changes from a 32-bit to a 64-bit integer format. A value of 3,000,000,000 remains exactly representable in that warehouse contract. Updating four local definitions suffices regardless of the number of users. An unchanged 32-bit public contract would not admit that example value.

Old batches retain their original decoder selection; the new decoder maps directly to D. Identity, payloads, hashdiffs, availability, historical data, and the shared gate remain unchanged. Hence R1–R7 survive. P18a treats canonical widening with a checked old interface; P21 treats replacement with historical-only old service.

2.19 P18a: Canonical Type Widening through a New Satellite

Request and premises. A source widens a descriptive Balance field from signed `INT32` to `INT64`, and new values may exceed the warehouse's old domain. Retain the original `balance_v1` read contract and introduce `balance_v2` for the full widened values. Unlike P18, this operation changes warehouse storage by adding a Satellite; it does not alter the old Satellite's payload, type, or hashdiff.

Assume R1–R7, unchanged business identity, units, grain, and observation semantics. The source capture already retains the full widened values, and the wider decoder, storage type, resolver, and controller are supported. The parent key and active table-feed bindings already exist. New mechanisms, identity changes, and business-defined capping or truncation are outside this recipe.

The existing version-aware `balance_v1` contract returns an exact `INT32` value when representable,

or R7's observation-specific unavailable result otherwise. All its consumers already understand this protocol; no new return type or diagnostic branch is imposed on them. Its provider binding is independent of the source-field acquisition binding. All retained live obligations for the old value are served through this one contract; additional live providers are counted separately.

Choose a cutover observation b_0. Complete the last accepted old observation and drain or cancel pending old invocations, including retries. Before publication, initialize the new Satellite from a complete captured observation at b_0, including unchanged balances. This establishes its own history without depending on rows in the old Satellite. Existing as-of interfaces provide correct values at completed boundaries, as assumed in P9.

Construction. Use P3's ten-artifact pattern for a new logical field `balance_64`, narrow Stage path, and `SAT_CUSTOMER_BALANCE_64`. Reuse the table feed and key bindings. The new loader retains full `INT64` values in the Raw Vault. The old `INT32` Satellite, its hashes, and its history remain unchanged.

At b_0, retire only the old field-acquisition binding using R7's two-artifact retirement pattern. Other source fields continue. The replacement path does not populate the old Satellite. This ends old acquisition, not the independently bound `balance_v1` service.

Add a checked compatibility provider for `balance_v1`: a Task, its read-interface definition, an effective provider record, and a local test. This follows P9's four-artifact provider pattern; the original public contract remains unchanged. Earlier observations retain the old provider. From b_0, the new provider reads the wide Satellite and checks representability in the old domain.

Finally, use P4's four-artifact pattern to publish the requested `balance_v2` delivery interface over the new Satellite's already-counted read port. This version returns full widened values for completed observations from b_0 onward. Providing pre-cutover history through v2 would be an additional, explicitly counted requirement; it remains accessible through v1 here.

Read-contract argument. Let the old and new value domains be

$$\begin{aligned} D_{32} &= \{x \in \mathbb{Z} : -2^{31} \le x \le 2^{31}-1\}, \\ D_{64} &= \{x \in \mathbb{Z} : -2^{63} \le x \le 2^{63}-1\}. \end{aligned} \tag{28}$$

Thus $D_{32} \subset D_{64}$. Let $\mathrm{Value}(x)$ denote an available typed value; supplied NULL and other unavailable states remain separate protocol outcomes. For a supplied $x \in D_{64}$, the new provider's checked projection is

$$C_{32}(x) = \begin{cases} \mathrm{Value}(\mathrm{int32}(x)), & x \in D_{32}, \\ \mathrm{Unavailable}(\texttt{outOfRange}), & x \notin D_{32}. \end{cases} \tag{29}$$

The cast is exact on D_{32}. Supplied NULL and prior unavailable outcomes are propagated according to their existing contracts, not passed to this numeric cast. The v2 projection returns $\mathrm{Value}(x)$ for every supplied $x \in D_{64}$.

Consequently every previously representable value remains exact through v1, and newly unrepresentable values produce an outcome already admitted by v1. Its integer domain has not been enlarged silently. R7 propagates unavailable required inputs through dependent rules without definition changes. This preserves the value-or-status contract, not an unconditional promise of a numeric result for every new source value.

For one customer, consider successive completed balances 1,000, 3,000,000,000, and 1,500 after cutover. The wide Satellite preserves all three. Version v2 returns all three exactly, while v1 returns $\mathrm{Value}(1000)$, unavailable, and $\mathrm{Value}(1500)$. At the middle observation, v1 must not fall back to 1,000. Its status is derived from the wide value valid at that observation, including during later historical reads. Other customers with representable balances remain available.

Artifact count and locality. Let W be the initially conforming warehouse, $N = |V|$ its artifact count, and $\Delta_{\text{canonical+}}$ this request. Its footprint $M(W, \Delta_{\text{canonical+}})$ counts added, changed, or retired definitions, not the number of loaded rows. For the specified one-field, one-old-interface, one-new-interface recipe,

$$\begin{aligned} M(W, \Delta_{\text{canonical+}}) &\le 10_{\text{new path}} + 2_{\text{old acquisition}} \\ &\quad + 4_{\text{v1 provider}} + 4_{\text{v2 delivery}} \\ &= K_{\text{canonical+}} = 20. \end{aligned} \tag{30}$$

The ten-artifact path includes the field binding, projection, Stage storage/access, Satellite storage/access, loader, two Flow registrations, and test. The retirement pair comprises a binding version and test. Each four-artifact provider or delivery unit comprises a Task, read-interface definition, binding, and test. Runtime capture, initial loading, and test execution do not alter additional definitions under the premises.

Both five and 500 existing v1 consumers require zero definition edits: they use the same provider selection and result protocol. The old field binding is

not consulted to determine the new provider's availability; its declared input is the active wide read port. No old loader, consumer, or hash definition is rewritten.

Preservation and limits. The new one-attribute Satellite and retained payloads preserve R1. Exact widened storage and the predefined checked old contract preserve R2; registered tasks, pinned observations, and gates preserve R3–R4. History and earlier providers remain under R5; independent source bindings preserve R6; both versions instantiate R7. Overflow is observation-specific, not global field retirement.

For L distinct old live contracts needing separate providers of this same bounded form, and a additional mechanism definitions, the bound becomes $16 + 4L + a$. Here $L = 1$ and $a = 0$. Accumulated live contracts are existing obligations: repeated widening has no uniform bound unless provider count and reconstruction size remain bounded. Contracts that never admitted out-of-range unavailability, unsupported capture formats, key-field widening, and arbitrary semantic conversions require separate reasoning and accounting.

2.20 P19: Narrow a Source Type on an Exactly Recoverable Domain

Request and premises. A source changes to a narrower representation, but every value required by the existing warehouse contract is still exactly recoverable on the admitted input domain. R1–R7 hold; no unit, identity, null, or time meaning changes. The admitted domain is specified independently of the warehouse's size.

Proof and construction. Let D be that canonical domain, $c : D \to S_c$ the contracted encoding, and $g : S_c \to D$ the exact decoder. The substantive premise is

$$g(c(x)) = x \quad (x \in D). \tag{31}$$

This equality preserves every existing consumer computation over D. Add one field/format mapping, one decoder Task, its one format registration, and one local test specification. Let N include existing artifacts and versions; Δ_{narrow} be the change; $M(W, \Delta_{\text{narrow}})$ count distinct artifact changes; and K_{narrow} be the bound:

$$M(W, \Delta_{\text{narrow}}) \leq 1 + 1 + 1 + 1 = K_{\text{narrow}} = 4. \tag{32}$$

Numerical example. The source previously represents whole-item quantities as decimals but guarantees values in $D = \{0, 1, \ldots, 1000\}$. It now sends integers. The decoder restores the same canonical whole-number quantity and canonical serialization. Four local definitions suffice for 10 or 1,000 dependent paths. No old Satellite data, hash recipe, delivery definition, Flow, or Trigger is changed.

Preservation and boundary. Source mappings and the decoder remain owned by the Connector; invocations pin their versions and check the existing feed gate. No payload or identity changes, no adapter chain forms, and history remains accessible. R1–R7 are preserved.

The restricted domain is essential. If contraction maps both 1.2 and 1.8 to 1, one decoder output cannot recover both. A successful cast would not establish compatibility. Such information loss requires a changed contract, retained additional information, or an explicitly permitted historical-only result; it is not included in the four-artifact bound.

2.21 P20: Recode Categories without Changing Their Meaning

Request and premises. The source replaces text category labels by integer codes while the warehouse retains the same logical categories. For example, `Open`, `Closed`, and `Pending` become codes 10, 20, and 30. R1–R7 hold. The source-approved mapping covers the complete admitted domain and preserves meanings; unknown values use the existing invalid/unavailable protocol.

Semantic argument. Let D be the canonical category set, $e : D \to S$ the new encoding and $g : S \to D$ its decoder. Require $g(e(x)) = x$ for each $x \in D$. Thus the former category, not merely a compatible scalar type, reaches every existing consumer and Satellite hash expression.

Construction and count. Add one field/format-binding version, the decoder Task, its format registration, one local test specification, and r independently specified value-mapping rules. Let N count existing artifacts including all previous maps; Δ_{recode} denote the change; $M(W, \Delta_{\text{recode}})$ count distinct artifact changes; and K_{recode} be the bound:

$$\begin{aligned} M(W, \Delta_{\text{recode}}) \leq\ & 1_{\text{mapping}} + 1_{\text{decoder}} + 1_{\text{binding}} \\ & + 1_{\text{test}} + r = K_{\text{recode}} = 4 + r. \end{aligned} \tag{33}$$

For three rules, $r = 3$ and $K_{\text{recode}} = 7$. A dictionary of 200 independent new correspondences has a larger input description and a larger bound; it is not one atomic edit because it occupies one file.

Locality and preservation. Data mappings and the decoder within the source Connector change. Canonical field identity, payloads, stored history, key/hash specifications, dependent Tasks, Flows, and Triggers remain. The decoder targets the canonical domain directly; old formats are retained for their batches. R1–R7 therefore remain true, independently of the number of downstream category users.

Merging two previously distinct categories, changing a grade's interpretation, or treating an unknown code as an invented category is not exact recoding. An approved new meaning can use a new versioned data construction, but that is a different request with different legacy obligations.

2.22 P21: Introduce a New Descriptive Domain or Category Meaning

Request and premises. A descriptive field acquires a value domain or category meaning that cannot satisfy its old live contract. Publish the new meaning while retaining the old field's history. The new domain, parent identity, and applicable decoder are explicitly approved. The parent Hub/Link identity is unchanged.

R1–R7 hold. Retire the old descriptive field using P12; its existing contract already permits the stated historical cutoff and unavailable-current result. R7 is a baseline requirement, not an additional premise. Existing consumers are not promised recovery of lost live values. A new field format unsupported by an existing decoder introduces additional counted definitions.

Construction and count. Give the new meaning a new logical field identifier, binding, one-attribute Satellite, and narrow acquisition path. Do not reuse the old canonical name to redefine its meaning silently. Retire the old field through its versioned field-level lifecycle, not by disabling the entire source table.

Let N count existing artifacts and versions; Δ_{domain} be this versioned replacement; and $M(W, \Delta_{\text{domain}})$ count distinct artifact changes. Let a count additional decoder/mapping definitions beyond the supported-format recipe, and d the number of explicitly requested simple delivery interfaces. Attribute addition costs 10, baseline field retirement costs 2, and each stated delivery addition costs 4. Therefore

$$\begin{aligned} M(W, \Delta_{\text{domain}}) &\leq 10 + 2 + a + 4d \\ &= K_{\text{domain}} = 12 + a + 4d. \end{aligned} \tag{34}$$

The numerical example $a = d = 0$ gives 12; one named new delivery interface gives 16. Existing Data structures and old task implementations are retained; new Data, Connector, Task and Flow artifacts belong to the new field path. Triggers are unchanged.

Why it remains local and preserves the rules. The old interface remains correct for its declared history and availability state, while only new or named consumers request the new meaning. The new loader never rewrites old payloads or hashdiffs. Independent logical mappings, source gates, and version-pinned invocations preserve R1–R7. The new field starts active with the same availability protocol as the retired field.

This is an additive replacement plus admitted retirement, not live DVT for an irreversible conversion. If the old field must remain live and semantically equivalent, its reconstruction must be proved separately; the mere existence of two Satellites does not discharge that obligation.

2.23 P22: Replace a Hub Business-Key Definition

Request and premises. A source-level entity changes its identifying key from `CustomerNo` to the pair (`Country`, `CustomerNo`). The request adopts a new identity contract, acquires the replacement facts, and retains the old identity's history. It does not require re-keying every existing Link, Satellite, or live consumer.

R1–R7 hold. The supplied replacement feed version covers the new business meaning and includes the selected fields and relationships. Existing contracts of the retired contribution permit historical-only access. Other source contributions and their existing identities remain valid. Any correspondence between the old and new identities is separately approved and counted; matching key strings alone is insufficient.

Construction and count. Register a new logical feed version for the replacement source contract, even if its physical table name is unchanged. Add the new Hub and its requested source-specific Satellites/Links using the table-addition recipe. Retire the old logical feed after a completed boundary, retaining its structures and mappings. Do not change the key meaning of the old Hub.

Let N count existing artifacts and versions; Δ_{key} denote the replacement; q count replacement load paths, $v \le q$ its new targets, and f its selected source fields. Let a be all additional identity-mapping or requested delivery artifacts. For distinct-artifact count M,

$$M(W, \Delta_{\text{key}}) \le (2 + f + 7q + 2v) + 2 + a = K_{\text{key}}. \quad (35)$$

With Country, CustomerNo, and Name, take $f = 3$, a new Hub and Name Satellite ($q = v = 2$), and $a = 0$. The bound is $2 + 3 + 14 + 4 + 2 = 25$.

Locality and preservation. The new Connector bindings, Data objects, Tasks, and Flows are those of the replacement model. The old lifecycle record changes, but old references still name the old Hub. No consumer is silently redirected to a different identity. The existing Trigger continues to process active feeds. The two component operations preserve R1–R7.

This proves versioned adoption with historical retention. Continuing live old identities needs an explicit identity reconstruction, as in P9. A mandatory global re-keying of all existing dependents is not covered by K_{key}.

2.24 P23: Add a Participant to a Relationship Version

Request and premises. Introduce a new Link describing (`OrderId`, `ProductId`, `SupplierId`) beside an existing Order–Product Link. All three Hub identity contracts already exist, and existing acquisition supplies every referenced Supplier identity before the new Link is published. The supplier field is newly selected from the same active feed and uses a supported decoder. Otherwise, an additional Supplier Hub path must be counted. No existing Link's participant list or hash recipe changes.

R1–R7 hold. The source guarantees that its projections required by old paths remain valid, including values, multiplicities, and observation times. For example, one supplier is supplied for each old order/product pair at an admitted observation boundary. The new participant cannot silently change the old row-level grain or descriptive meaning. More general decompositions require a separate compatibility construction.

Construction and count. Add one logical field binding, a new relationship projection, Stage storage/read interface, a new Link storage/read interface, one loader, two flow registrations, and one local test specification. Reuse the feed's gate, relation binding, existing key-field mappings, and Hub ports.

Let N count existing artifacts and versions; $\Delta_{\text{link+}}$ denote this new relationship version; $M(W, \Delta_{\text{link+}})$ count distinct artifact changes; and $K_{\text{link+}}$ be its bound:

$$\begin{aligned} M(W, \Delta_{\text{link+}}) &\le 1_{\text{field}} + 1_{\text{projection}} + 2_{\text{Stage}} + 2_{\text{Link}} \\ &\quad + 1_{\text{loader}} + 2_{\text{flow}} + 1_{\text{test}} \\ &= K_{\text{link+}} = 10. \end{aligned} \quad (36)$$

If the field binding already exists the corresponding term is zero. New Hubs or descriptive Link Satellites add their separately specified paths and structures.

Proof and preservation. The old loader still consumes its fixed old projection. The new loader records the explicitly supplied three-party relationship rather than inferring it from a Link-on-Link structure. Its three-identity grain belongs to a new data contract, so existing readers are not repointed to it.

Data and Connector artifacts grow additively; the new Task and Flow use the same state and write-coordination protocols. The Trigger, old hashes, old relationships, and old consumers remain unchanged. Thus the bound is independent of the old relationship's number of readers, and R1–R7 remain valid. The old-projection premise is the compatibility proof boundary, not a consequence of creating a new Link name.

2.25 P24: Remove a Participant from a Relationship Version

Request and premises. Replace a three-party Order, Product, and Supplier relationship by a new Order–Product relationship. Both remaining participants have existing Hub contracts. The supplied new contract defines distinct pairs as its grain; it does not preserve old triple multiplicities by renaming the old Link.

R1–R7 hold. If Supplier remains available, the old relationship may continue unchanged. If its source field is physically withdrawn, R7 retires that logical binding: old paths requiring it stop and their existing contracts admit retained history. The remaining keys still resolve correctly. Unlike descriptive withdrawal, this removes a relationship role; the approved new Link and the old triple contract's lifecycle are additional semantic premises of this proof, not guarantees supplied by R7.

Construction and count. Add one path and a new two-party Link using existing field bindings and the

active feed. Its seven path artifacts are a Connector projection, Stage storage/interface, loader, two Flow registrations, and test. Its two target artifacts are Link storage/interface. When the source removes the role field, append its retired field-binding version and its local test.

Let N count existing artifacts including versions; $\Delta_{\text{link}-}$ denote this operation; $M(W, \Delta_{\text{link}-})$ count distinct artifact changes; and $u \in \{0,1\}$ indicate physical field withdrawal through R7.

$$M(W, \Delta_{\text{link}-}) \leq 7_{\text{path}} + 2_{\text{Link}} + 2u = K_{\text{link}-} = 9 + 2u. \tag{37}$$

Thus the new coarser relationship costs at most 9 when the old feed remains fully available, or 11 with the stated field withdrawal. Additional mappings or new endpoint Hubs are extra.

Semantic boundary and preservation. For an available old relation R of triples, the new relationship set may be specified as

$$S = \{(o,p) : \exists s \ (o,p,s) \in R\}. \tag{38}$$

Eliminating repeated pairs is an intentional property of this new contract, not an exact inverse that recovers all triples. When the new source no longer provides s, no current old triple is fabricated. Old triple history remains as history.

New Data, Task, Connector and Flow definitions are local; old identities, payloads, hashes and the common Trigger remain. The new path inherits active-field readiness; the old path respects the retired binding. R1–R7 remain valid. A requirement for continued current triple information without a supplier source falls outside this proof.

2.26 P25: Split an Entity through Table Addition and Retirement

Request and premises. Replace a source `CLIENT` table by `PERSON` and `ACCOUNT`. Acquire the supplied replacement model and retain Client history; continued live reconstruction of the old Client representation is not required. R1–R7 hold, the old contract admits retirement, and the new keys, fields, relationships, and source correspondence are explicitly approved.

A business entity is not assumed to be one table. This operation is stated in terms of the actual two replacement feeds and their required Vault mappings. Any relationships required by the Account model are included in its acquisition specification.

Construction and counting. Integrate Person, then Account when its required identities and ports are available, using the source-table addition recipe. After the agreed completed transition boundary, retire the old Client feed through its lifecycle gate. Do not re-key its old Hub or remove its descriptive history.

Let N count all existing and retained artifacts, Δ_{splitNew} be this new-model adoption, and $M(W, \Delta_{\text{splitNew}})$ count distinct artifact changes. For replacement feed $i \in \{P,A\}$, let f_i count selected fields, q_i load paths, and v_i newly created Vault objects. Set $K_i = 2 + f_i + 7q_i + 2v_i$. Then

$$M(W, \Delta_{\text{splitNew}}) \leq K_P + K_A + 2 = K_{\text{splitNew}}. \tag{39}$$

The final two artifacts are the old feed's control update and retirement-test specification. Shared artifacts can reduce the sum; required mapping logic cannot be omitted.

Numerical example. Person has Id and Name, with a new Hub and Name Satellite: $f_P = q_P = v_P = 2$, so $K_P = 22$. Account has Id, PersonId, and Balance, with a new Hub, Person–Account Link, and Balance Satellite: $f_A = q_A = v_A = 3$, so $K_A = 32$. Thus $K_{\text{splitNew}} = 22 + 32 + 2 = 56$. New delivery interfaces are additional requests. This example supplies no LegacyClientId and does not claim a cross-version identity correspondence. Old Client observations remain accessible through their retained provider; the new feeds do not populate the old Satellites.

Locality and preservation. The new Data, Connector, Task and Flow definitions belong to the two specified feeds; their source bindings are explicit. Existing Triggers, old Client structures, and unrelated consumers are unchanged. Each step preserves R1–R7 and supplies the next step's prerequisites, so the finite composition is conforming. This is not the 77-artifact live-legacy construction of P9. A contract promising fresh old Client meaning requires that stronger reconstruction argument, not merely retained tables.

2.27 P26: Horizontally Split a Source while Preserving a Live Read Contract

Request and semantic premises. A source replaces Customer(Id, Name) by two tables with the same fields, for example fixed North and South customer partitions. Keep one specified live customer read contract. R1–R7 hold, both partitions have coherent completed observations, and the source approves their identity and value correspondence. The

partitions are complete and disjoint at each observation. For the worked example, identities are only appended and each identity's partition membership remains fixed; row withdrawal or reassignment needs additional temporal handling.

Let D_b be the bag required by the old contract at observation boundary b, and P_b, Q_b the replacement bags. Require

$$D_b = P_b \uplus Q_b, \qquad \text{keys}(P_b) \cap \text{keys}(Q_b) = \varnothing. \quad (40)$$

Here $\uplus$ is bag union: it adds multiplicities rather than silently eliminating duplicates. These are semantic premises, not conclusions inferred from table names.

The snapshot-to-Vault interpretation is subject to the explicit read-realization premise in Section 1.4.

Construction and bound. Integrate both partitions as new feeds with source-specific Satellites and the approved shared Hub identity. Add one on-demand compatibility Task forming the bag union, its Data read contract, one effective provider binding, and a local test specification. The existing version-aware delivery mechanism selects this provider for the new observation interval. Retire the old feed through its gate after a completed transition.

Let N count existing artifacts and versions; Δ_{hsplit} be the request; M its distinct artifact changes; and $K_i = 2 + f_i + 7q_i + 2v_i$ the bound for partition i with its selected fields, paths, and new targets. Then

$$M(W, \Delta_{\text{hsplit}}) \le K_P + K_Q + 4 + 2 = K_{\text{hsplit}}. \quad (41)$$

For each partition, $f_i = 2$, $q_i = 2$, $v_i = 1$: a reused Hub path and a new Name Satellite path. Each costs 20, giving $K_{\text{hsplit}} = 20 + 20 + 4 + 2 = 46$.

Why the result is local and preserved. The union equals the declared old bag, including its identity, values, and multiplicities. Its old public contract therefore needs no consumer repair. Earlier observations use retained providers. Fresh reads use coherent replacement observations, not arbitrary latest versions or the union of every row ever loaded. New Tasks use stateful invocation; the old source is gated; old history remains; names resolve through R6. This proves R1–R7 preservation for the stated two-partition case. Incomplete partitions, overlapping memberships, or unavailable legacy fields do not satisfy the equality and are not covered.

2.28 P27: Merge Source Partitions while Preserving Their Live Read Contracts

Request and premises. Two previously separate North and South customer feeds become one source table Customer(Id, Name, Region). Keep the two specified old live read contracts. R1–R7 hold; identities do not collide across partitions, Region is a source-supplied discriminator, and its two fixed meanings are approved. Membership remains fixed per identity in the worked case, with no row-level withdrawals. Observation boundaries and field meanings are preserved.

For a merged bag U_b at boundary b, let p_N and p_S be the two supplied, disjoint and exhaustive predicates. Let σ_p filter a bag and π project the old fields without duplicate elimination. Require

$$D_{N,b} = \pi(\sigma_{p_N}(U_b)), \qquad D_{S,b} = \pi(\sigma_{p_S}(U_b)), \quad (42)$$

where the two D bags are the results promised by the old interfaces.

The snapshot-to-Vault interpretation is subject to the explicit read-realization premise in Section 1.4.

Construction and count. Acquire the merged feed. Create a separate bounded compatibility projection for each old contract; each uses the appropriate Region predicate and a direct effective provider binding. Retire both old feeds after the coherent cutover and keep their histories and earlier providers.

Let N count existing and retained artifacts; Δ_{hmerge} be this merge; M count distinct artifact changes; and $K_U = 2 + f + 7q + 2v$ count acquisition of the supplied merged mapping. Each of the two read providers has four artifacts: Task, Data contract, binding, and local test. Each old feed retirement has two:

$$M(W, \Delta_{\text{hmerge}}) \le K_U + 2(4) + 2(2) = K_{\text{hmerge}}. \quad (43)$$

For $f = 3, q = 3, v = 2$ (reused Customer Hub, new Name and Region Satellites), $K_U = 30$ and $K_{\text{hmerge}} = 42$.

Locality and preservation. The two equalities establish exactly the old supported populations, not merely a shared column layout. Their users keep unchanged interfaces, so neither consumer count nor unrelated source count appears. R1–R7 are preserved by the new additive Data, Connector, Task, and Flow construction, explicit two provider records, stateful coherent reads, retained history, and the existing source gates and Trigger.

The example supports two fixed legacy contracts. A merge that forces separate reconstruction of an unbounded collection of old contracts is not this fixed-size request. Missing provenance or overlapping identity meanings cannot be repaired by the counting argument.

2.29 P28: Vertically Split a Source while Preserving a Live Read Contract

Request and premises. Customer(Id, Name, City) becomes PersonName(Id, Name) and PersonCity(Id, City). Preserve the specified old live customer contract. R1–R7 hold. At every supported completed observation, Id is unique in each part, the parts contain exactly the same required identity set, and field meanings, null rules, and time interpretation are unchanged. The source approves this lossless decomposition. In the example, identities are only appended; unmodeled row deletions or key reassignment are excluded.

For one observation b, write L_b, R_b for the two bags, D_b for the old promised bag, and G for the join and projection onto Id, Name, City. The premises imply

$$G(L_b, R_b) = \big[(i,n,c) : (i,n) \in L_b,\ (i,c) \in R_b\big] = D_b. \tag{44}$$

The brackets denote a bag. Each key has exactly one partner, so the join neither loses nor multiplies rows.

The snapshot-to-Vault interpretation is subject to the explicit read-realization premise in Section 1.4.

Construction and bound. Acquire both new feeds using the same approved existing Hub identity and separate source-specific Satellites. Add an on-demand compatibility Task, read contract, effective provider binding, and local test. Select its coherent inputs by observation boundary. Retire the old feed, retaining earlier providers.

Let N count existing artifacts including versions; Δ_{vsplit} be this change; M its distinct artifact changes; and $K_i = 2 + f_i + 7q_i + 2v_i$ be the acquisition bound for part i. Then

$$M(W, \Delta_{\text{vsplit}}) \le K_L + K_R + 4 + 2 = K_{\text{vsplit}}. \tag{45}$$

Each part uses two fields, two paths (reused Hub plus new Satellite), and one new target, so $K_L = K_R = 20$ and the worked bound is 46.

Preservation and boundary. The equality supplies the semantic reason old consumers need no change. Only the declared new Data, Connector, Task, Flow and compatibility definitions change; the Trigger and existing schemas remain. State, history, source gates, and independent physical mappings preserve R1–R7.

A join on a non-unique key, an incomplete part, or mismatched observations may fail to reconstruct D_b. Creating two tables and writing a join is therefore not a general losslessness proof. The present example is a restricted data-preserving specialization, not arbitrary domain decomposition.

2.30 P29: Join Source Structures while Preserving Their Live Read Contracts

Request and premises. The separate PersonName(Id, Name) and PersonCity(Id, City) feeds become Customer(Id, Name, City). Continue the two old live read contracts. R1–R7 hold. Id is unique, the required key sets match, the joined source loses no required row, and meanings, nulls, and observation boundaries remain unchanged. The source approves the correspondence. The simple example has no unmodeled identity reassignment or row-level withdrawal.

Let $D_{L,b}, D_{R,b}$ be the promised old bags at boundary b and U_b the new joined bag. With bag projections onto the named fields, require

$$\pi_{\text{Id,Name}}(U_b) = D_{L,b}, \qquad \pi_{\text{Id,City}}(U_b) = D_{R,b}. \tag{46}$$

Keeping the key in both projections and the uniqueness premise preserves multiplicity. These equalities are not implied by an arbitrary inner join of the original sources.

The snapshot-to-Vault interpretation is subject to the explicit read-realization premise in Section 1.4.

Construction and count. Acquire the combined feed through the existing Hub identity and new Name and City Satellite histories. Introduce two on-demand compatibility providers, each with a Task, read contract, effective provider record, and local test. Earlier observations continue through old providers. Retire both old feeds after the coherent new inputs are available.

Let N count all existing and retained artifacts; Δ_{vjoin} denote the request; M count distinct artifact changes; and $K_U = 2 + f + 7q + 2v$ count the new feed mapping. Then

$$M(W, \Delta_{\text{vjoin}}) \le K_U + 2(4) + 2(2) = K_{\text{vjoin}}. \tag{47}$$

For $f = 3, q = 3, v = 2$ (reused Hub and two new Satellites), $K_U = 30$ and $K_{\text{vjoin}} = 42$.

Locality and preservation. The old read contracts remain correct by the two projections. Their consumers do not discover the physical join or acquire new fields automatically. New Data, Connector, Task, and Flow definitions and the four-artifact providers form a fixed region; source retirement uses existing gates. Immutable payloads, version-pinned state, historical providers, source indirection, and the unchanged Trigger preserve R1–R7.

If the source join discarded rows without a partner, the relevant equality can fail. Additional preserved

information or another mapping is then required and must be counted. The proof does not establish arbitrary joining as a transparent migration.

2.31 P30: Change Source Constraints while Retaining the Canonical Contract

Restricted request and premises. A source changes an enforcement declaration, but continues to satisfy every invariant required by the warehouse's existing canonical contract. For example, it adds a NOT NULL declaration to a field already contractually guaranteed non-null. No required row is removed, no admitted identity changes, and the warehouse does not derive its logical schema from source constraint names.

R1–R7 hold. The resolver checks the required data/interface properties, not equality with a copied complete constraint list. The constraint change neither invalidates an existing mapping premise nor requires new warehouse validation functionality. These strong conditions distinguish this restricted case from general constraint evolution.

Construction and proof. Keep the source bindings, canonical data structures, Tasks, Flow definitions, and Trigger unchanged. Add one local conformance-test specification for the changed source guarantee. Let N count existing and retained artifacts; $\Delta_{\text{constraint}}$ be this restricted enforcement change. The number of warehouse artifact changes is $M(W, \Delta_{\text{constraint}})$; its recipe bound is $K_{\text{constraint}}$:

$$M(W, \Delta_{\text{constraint}}) \leq 1_{\text{local test}} = K_{\text{constraint}} = 1. \quad (48)$$

The source's own DDL or equivalent configuration is outside the warehouse boundary. If the existing parameterized tests need no new specification, the actual structural count can be zero.

Example and preservation. A source formally enforces the previously agreed non-null CustomerId invariant. Fifty existing extraction paths still receive the same admitted identifier population and call the same canonical port. One new local test, rather than fifty model changes, suffices in the stated recipe. No new Data, Connector, Task, Flow, or Trigger definition is required, and R1–R7 remain true.

This is a no-propagation special case, not a general guarantee for uniqueness, nullability, or foreign-key revisions. Dropping the only uniqueness guarantee while duplicates become possible, changing key meaning, or rejecting formerly required rows violates the premise. Re-establishing validity, adding information, or changing supported contracts needs a separate counted construction. Successful schema compilation is not evidence that those semantic conditions hold.

3 COMPOSITION AND LIMITS

3.1 P31: Preservation across Successive Admitted Changes

Use $W = (V, E, \kappa, d, \gamma)$ and $\mathcal{A}$ from Section 1.3. Each step must satisfy its own $\text{Pre}_{\tau_t}(W_{t-1}, \Delta_t)$, including the semantic and coordinated-retirement premises. The following sequence argument applies only to admitted cases.

Corollary 1 (Successive admitted changes)**.** *Let $W_0 \in \mathcal{A}$. At step t, applying the recipe for change Δ_t produces $W_t = T_{\Delta_t}(W_{t-1})$. If each step satisfies its premises, then $W_t \in \mathcal{A}$ and, for any finite sequence of k operations,*

$$\sum_{t=1}^{k} M(W_{t-1}, \Delta_t) \leq \sum_{t=1}^{k} K_{\tau_t}(s(\Delta_t)). \quad (49)$$

Proof. For P1–P9, Theorem 1 gives preservation and a bound at each admitted step. For P10–P30, including P18a, use the corresponding conditional construction and its preservation argument, including all additional premises. Induction then establishes conformance throughout, and summing the change-specific bounds gives the inequality. This does not establish the premises of an arbitrary future request. □

A uniform per-change bound requires a common upper scope s_0, giving $K_*(s_0) = \max_\tau K_\tau(s_0)$ over the included recipes. Counts and description sizes of loading paths, fields, value mappings, rule inputs and parameters, and reconstructions must remain bounded independently of warehouse size and version history. Thus, 12 and 77 bound the worked cases, not arbitrary changes. Direct adapters do not ensure uniformity if their number or complexity grows. Preserving R1–R7 does not guarantee that every later change is admissible. For example, a delivery interface that acquires an internal dependent task can no longer be retired using P5's leaf-interface recipe without addressing that dependency.

Preservation mechanisms. Additions initialize immutable payloads, logical source bindings, and input-availability contracts. Retirement interprets existing lifecycle states; compatible representation, policy,

and recovery changes preserve their data and action interfaces. New rules apply R7 to both raw and derived inputs, so a later withdrawal needs no consumer-specific branches. P18a adds a wider Satellite without rewriting its predecessor; checked old interfaces derive each observation's range status from the wide value, preserving R7 without consumer-specific edits. The restricted reconstructions additionally preserve their stipulated identities, multiplicities, values, and observation boundaries.

Example and scope. Adding email (10), renaming its source table (2), adding phone (10), withdrawing email (2), and retiring the feed (2) has total bound 26. Both new paths retain the protocols needed by subsequent operations. Phone remains available after email withdrawal until feed retirement. For a fixed upper scope, each admitted step is bounded by $K_*(s_0)$; a k-step sequence is bounded by $kK_*(s_0)$, not one constant independent of k.

Repeated split–merge operations do not authorize rewriting every historical identity or reconstructing an increasing set of live contracts at no cost. For P9, the Client Hub represents the retained identity; its reused loading path does not populate the old Client Satellites. Required additional providers or more complex reconstructions must appear in subsequent footprints. P18a likewise counts separate checked providers for retained live contracts; repeatedly widening a type cannot hide a growing provider set. P14 remains proportional to its selected feed count.

The left side of Equation 49 sums per-step impacts; an artifact modified at two different steps contributes at both. Counting distinct artifacts over the complete sequence can be smaller, but cannot exceed that sum. Neither measure bounds runtime, semantic-analysis effort, or data reprocessing.

3.2 Boundary Result 1: Lost Information Cannot Be Reconstructed Automatically

Statement and definitions Let X be a set of possible source observations, $h : X \to Y$ an information-losing source transformation, and $f : X \to Z$ the output required by an old live contract. Suppose two admitted observations $x, y \in X$ satisfy

$$h(x) = h(y), \qquad f(x) \neq f(y). \tag{50}$$

Then no adapter $g : Y \to Z$ using only the transformed observation can satisfy $g(h(z)) = f(z)$ for every admitted z.

Proof The same adapter input would require $g(h(x)) = f(x)$ and $g(h(y)) = f(y)$. Since $h(x) = h(y)$, the left sides are equal but the right sides are not, a contradiction. For example, rounding both 1.2 and 1.8 to 1 cannot preserve a contract returning the original decimal when only that integer remains.

Implication for the catalogue A narrowing conversion, removed Link participant, discarded partition, or changed category may meet this condition. Additional retained information, another live source, or a changed contractual promise is then necessary. Historical data does not determine an unobserved future value. This result does not rule out the additive/historical constructions above; it prevents presenting them as unconditional live compatibility.

3.3 Boundary Result 2: Forced System-Size-Dependent Repairs

Statement and definitions Let r be the number of distinct existing warehouse definitions that must each be revised under a particular implementation and fixed requirement Δ. Let N count all existing artifacts and $M(W, \Delta)$ the required distinct modifications. If these are genuinely unavoidable in that implementation, then

$$M(W, \Delta) \geq r. \tag{51}$$

If r can grow independently of the requirement's fixed scope, no constant K_Δ independent of warehouse size bounds it.

Proof and examples Each required definition is a distinct member of the changed set, so its cardinality is at least r. As r becomes arbitrarily large, any proposed fixed K_Δ is eventually exceeded. An implementation with r separately embedded physical names and no usable shared binding can exhibit this behavior on rename. Maintaining r distinct incompatible live legacy meanings may likewise force r separate reconstruction changes. Cloning paths or generating the edits does not remove the count.

Implication for the catalogue The local recipes avoid particular propagation mechanisms through construction; they do not prove that every warehouse or arbitrary semantic change can use those recipes. A source/target constraint revision that invalidates their identity, uniqueness, nullability, or completeness premises has no general guarantee here. Explicitly requesting r independent new products is different: the input itself then has size r.

AI-USE DISCLOSURE

ChatGPT assisted with drafting, shortening, and language editing. The authors remain responsible for the sources, assumptions, proofs, and final text.

REFERENCES

Awiti, J., Wrembel, R., and Zimányi, E. (2025). Repairing ETL Processes using Extended Relational Algebra. *Foundations of Computing and Decision Sciences*, 50(2):157–190.

Cuba, P. (2020). *The Data Vault Guru: a pragmatic guide on building a data vault*. Independently published, first edition edition.

Herrmann, K., Voigt, H., Pedersen, T. B., and Lehner, W. (2018). Multi-schema-version data management: data independence in the twenty-first century. *The VLDB Journal*, 27(4):547–571.

Linstedt, D. and Olschimke, M. (2016). *Building a scalable data warehouse with Data Vault 2.0*. Morgan Kaufmann, Waltham, Massachusetts. OCLC: ocn944715438.

Mannaert, H., Verelst, J., and De Bruyn, P. (2016). *Normalized systems theory: from foundations for evolvable software toward a general theory for evolvable design*. nsi-Press powered bei Koppa, Kermt.

Manousis, P., Vassiliadis, P., and Papastefanatos, G. (2015). Impact Analysis and Policy-Conforming Rewriting of Evolving Data-Intensive Ecosystems. *Journal on Data Semantics*, 4(4):231–267.

Rönnbäck, L., Regardt, O., Bergholtz, M., Johannesson, P., and Wohed, P. (2010). Anchor modeling — Agile information modeling in evolving data environments. *Data & Knowledge Engineering*, 69(12):1229–1253.

Vassiliadis, P., Shehaj, F., Kalampokis, G., and Zarras, A. V. (2023). Joint Source and Schema Evolution: Insights from a Study of 195 FOSS Projects. In *Proceedings of the 26th International Conference on Extending Database Technology (EDBT 2023)*, pages 27–39, Ioannina, Greece. OpenProceedings.org.